

The one feature: a working contact form

What a backend is

The site you see is just files — HTML, CSS, a bit of JavaScript — sitting on a server somewhere, handed to your browser exactly as written. Every visitor gets the identical copy. Those files can't remember anything, because nothing is running: they're documents, not programs.

A **backend** is a program that stays running on a computer you don't own, waiting to be asked something. It can do the things a document can't — receive data, store it, look things up, send an email. Your browser (the "frontend") talks to it by sending a request over the internet, and the backend sends an answer back.

The key thing I understood building this: my site had no backend at all. It was a poster. A form needs *something running* to catch what gets typed in, because a static file can't catch anything.

What my feature does

There's a contact form at the bottom of my site. Someone types their name, email, company, and what they're building, hits **Send message**, and that message lands in my inbox at `elemenx93@gmail.com` — plus in a dashboard where every submission is stored.

Before this, my only contact option was a `mailto:` link. That looks like it works, but it only does if the visitor's computer has an email program set up and connected. On a work laptop with webmail, clicking it often does nothing at all — the one action my whole site exists to drive was silently failing for some of the people I most wanted to reach. The form doesn't depend on anything installed on their machine.

How the data flows

1. **Someone fills in the form and clicks Send.** Nothing has left their browser yet — the text is just sitting in the page.
2. **My JavaScript intercepts the submit.** Normally a form submission makes the browser navigate away to a new page. I call `preventDefault()` to stop that, so the visitor stays where they are and I can show them a result inline instead of throwing them onto a different screen.
3. **The browser packs the fields up and sends them.** I use `fetch()` to POST the data back to my own site's address. "POST" is the request type that means *here is some data, take it* — as opposed to GET, which means *give me a page*. The fields are encoded as `name=Jane&email=jane%40company.com&...`, which is the same format a normal form submission uses.
4. **Netlify catches it before it reaches a file.** This is the part I had to get my head around, because there's no server of mine anywhere. When I deploy, Netlify reads through my HTML looking for any form tagged `data-netlify="true"`. Finding one, it registers a form called `contact` and starts listening for POSTs to my site. So Netlify's own servers are the backend — I didn't write one, I borrowed theirs. The hidden `form-name` field is how a POST identifies *which* registered form it belongs to; without it Netlify wouldn't know what the data was for.
5. **Netlify stores it and emails me.** The submission is saved in my site's Forms dashboard, and a notification goes to `elemenx93@gmail.com`. The storing and the emailing are separate — even if the email never arrives, the message is still recorded.
6. **The response comes back and I tell the visitor.** My JavaScript checks whether the response came back OK. If yes, it clears the form and shows "Thanks — that reached me." If the request failed, it shows my email address directly so the person isn't stuck with a dead end.

Two things I deliberately built in

It survives JavaScript failing. If my script has a bug, or someone blocks scripts, the `<form>` still has a real `method="POST"` and `action="/thanks.html"` on it — so the browser just submits it the old fashioned way, full page

navigation, and lands on my thank-you page. Netlify receives it identically either way. The JavaScript only makes the experience nicer; it isn't what makes the feature work.

Spam trap. There's an extra field called `bot-field` positioned off screen. A real person never sees it, so it stays empty. Automated spam bots fill in every field they find, so if that one arrives with anything in it, Netlify discards the submission. I declare it with `netlify-honeypot="bot-field"` so Netlify knows which field is the trap.

Cost

Free tier: 100 submissions per month, which is far more than a personal site gets. No server of mine is running, so there's nothing to pay for or keep alive.